



UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

COS 226 - Concurrent Systems

Assignment 1 Specification

Concurrent Auction and Lock Performance

Release Date: 25 August 2026

Submission Deadline: 20 September 2026

1 Overview

In this practical, you will implement and investigate three different mutual exclusion algorithms: TTAS, CLH, and MCS locks.

You will use these locks to synchronise a concurrent auction system in which multiple bidder threads compete to place increasingly higher bids on a shared auction item. The auction provides a common workload in which the different locks can be subjected to increasing levels of contention.

The practical consists of three main tasks. You will first implement TTAS, CLH, and MCS and establish that they provide correct mutual exclusion. You will then use your locks to construct a concurrent auction involving multiple bidder threads. Finally, you will design and conduct an experiment to compare the behaviour of TTAS, CLH, and MCS under increasing levels of contention.

The emphasis of this practical is on understanding the differences and trade-offs between the three lock implementations. Your experimental results should be used to investigate how the locks behave as contention increases, rather than simply determining which implementation has the lowest execution time.

2 Learning Outcomes

By completing this practical you should be able to:

- Implement TTAS, CLH, and MCS locks.
- Explain how TTAS, CLH, and MCS provide mutual exclusion.
- Explain the fundamental differences between TTAS, CLH, and MCS.
- Explain the purpose of queue-based locking in CLH and MCS.
- Identify the effects of increasing contention on concurrent programs.

- Analyse experimental results and identify meaningful differences between TTAS, CLH, and MCS.

3 Requirements

This practical is to be completed in groups of **three**. Group members are expected to contribute equally, and all group members should be able to explain any part of the solution during the demonstration.

A skeleton implementation will be provided containing an **Auction** class, an **AuctionUtils** utility class, a **Lock** interface, and a **Runner** class.

The provided **Runner** is responsible for creating, starting, and joining the bidder threads. The behaviour of each bidder and the experimental measurements are intentionally left to you.

You may structure your solution in any reasonable way, provided that your implementation satisfies the requirements of this practical.

You may not use Java's built-in locking mechanisms, including **synchronized**, **ReentrantLock**, or other high-level synchronisation utilities to implement your locks.

4 Task 1 – Lock Implementations

Implement the following three mutual exclusion algorithms:

- Test-Test-and-Set (TTAS)
- CLH Lock
- MCS Lock

Each implementation must conform to the provided **Lock** interface.

Your implementations must:

- Provide mutual exclusion.
- Allow one thread at a time to enter the critical section.
- Correctly release the lock when **unlock()** is called.
- Be safe for use by multiple concurrent threads.
- Not use Java's built-in locking mechanisms to provide mutual exclusion.

Your implementations should first be tested independently of the auction system to establish that they provide correct mutual exclusion.

5 Task 2 – Concurrent Auction

You will use your locks to implement a concurrent auction involving multiple bidder threads. The provided **Auction** class represents an auction for a single item. It maintains:

- The name of the item.
- The current highest bid.
- The identifier of the bidder who placed the highest bid.

Each bidder thread must repeatedly participate in the auction for the specified number of iterations.

The bidding behaviour is intentionally left open to you. However, each bidder must attempt to place a bid that is higher than the current highest bid.

The operation of determining the current highest bid and updating the auction must be correctly synchronised.

For example, the following sequence represents a logical bidding operation:

1. Obtain the current highest bid.
2. Determine a new bid that is higher than the current bid.
3. Submit the new bid to the auction.

You must determine which parts of this operation require mutual exclusion and ensure that your implementation prevents race conditions.

The auction must be shared by all bidder threads. Each thread must perform the same number of bidding iterations.

The provided **Runner** may be used to create and execute the bidder threads, but you are responsible for implementing the bidder behaviour.

6 Task 3 – Experimental Investigation

Design and conduct an experiment to investigate the behaviour of your three lock implementations under increasing levels of contention.

Increasing the number of bidder threads increases the number of threads competing for the same lock and therefore increases contention.

Your experiment must vary the number of concurrent bidder threads. At a minimum, your experiment must include:

- 2 threads
- 4 threads
- 8 threads
- 16 threads

You may investigate additional configurations if you believe they provide useful information.

For each thread configuration, all three lock implementations must perform the same workload.

The number of bidding iterations must therefore be kept constant when comparing the locks.

Each configuration must be executed at least **three times** for each lock. You should use the repeated runs to reduce the influence of random variation and background system activity.

6.1 Measurements

You must record several measurements during your experiment to allow you to compare the behaviour of TTAS, CLH, and MCS.

For every combination of lock and number of threads, you must record the following measurements:

- **Total execution time:** The total time taken for all bidder threads to complete the experiment.

- **Total number of bids:** The total number of bids successfully placed during the experiment. This should be used to verify that the expected workload was completed.
- **Final highest bid:** The value of the highest bid when all bidder threads have completed.
- **Bids won by each bidder:** The number of times each bidder was responsible for placing the highest bid. Use this measurement to investigate whether access to the auction was distributed evenly between the threads.

In addition to the measurements above, you must select and record **one additional measurement** that you believe provides useful information about the behaviour of the locks.

You must explain why you selected this measurement and how it contributes to the comparison of TTAS, CLH, and MCS.

Possible additional measurements include, but are not limited to:

- Average time spent waiting to acquire the lock.
- Maximum time spent waiting to acquire the lock.
- Number of successful lock acquisitions per thread.
- Difference between the bidder with the most bids and the bidder with the fewest bids.
- Another measurement that you can justify as relevant to the behaviour of the locks.

All measurements must be recorded for each lock and for each tested number of threads. Where appropriate, repeated runs should be averaged before comparing the results.

Your results must be presented clearly using tables.

7 Demonstration

Students are required to demonstrate their implementation to a tutor during their scheduled practical session.

During the demonstration, students must:

- Demonstrate that each lock provides mutual exclusion.
- Demonstrate the concurrent auction.
- Demonstrate the use of the three lock implementations.
- Explain the design of their bidder implementation.
- Explain the metrics selected for the experiment.
- Present and explain their experimental results.
- Justify the conclusions drawn from the experimental evidence.

Students should be prepared to explain their implementation and experimental results rather than simply demonstrate that the program produces the expected output.

Conceptual questions relating to mutual exclusion, contention, TTAS, CLH, MCS, fairness, and the behaviour of concurrent systems will be asked during the demonstration.

8 Upload Checklist

Before submitting, ensure that your group has included all of the following:

- `TTASLock.java`
- `CLHLock.java`
- `MCSLock.java`
- Your auction/bidder implementation.
- `Main.java`
- Experimental results.

You may include additional source files where appropriate.

9 Submission

Your solution will be submitted via **ClickUp**. **All group members are required to submit.** Your submission should include:

- All source code required to compile and run your solution.
- The student numbers of all group members.
- The experimental results.
- Your analysis and conclusions.

All required source files must be placed in a single ZIP archive. The archive must be named according to the following format:

`uXXXXXXXX.zip`

where `XXXXXXXX` is your student number.

No late submissions will be accepted!

10 Marking Rubric

Assessment Item	Marks
TTAS Lock implementation	2
CLH Lock implementation	2
MCS Lock implementation	2
Concurrent auction implementation	3
Experimental investigation and results	2
Conceptual Question 1	2
Conceptual Question 2	2
Conceptual Question 3	2
Total	17